

Maximum XOR With Value at Most M

Company: Google

Round: Online Assessment (OA) – SWE Intern

Year: 2022

Difficulty: Hard

Topic: Trie, Bitwise XOR, Offline Queries, Sorting

Source: [Leetcode - 1707](https://leetcode.com/problems/maximum-xor-with-value-at-most-m/)

Problem Description

- You are given an array of N integers and q queries. Each query gives two integers x and m . For each query, find an array value $arr[i]$ (with $arr[i] \leq m$) such that $x \oplus arr[i]$ is **maximized**, where $\oplus$ denotes bitwise XOR. If no array element satisfies $arr[i] \leq m$, print -1.

Input Format

- The first line contains an integer T — the number of test cases.
- For each test case:
 - A line containing integer N — the number of elements in the array.
 - A line with N space-separated integers $arr[1], arr[2], \dots, arr[N]$.
 - A line containing integer q — the number of queries.
 - q lines follow, each with two integers x and m .

Output Format

- For each test case, print q lines — the required maximum-XOR answer for each query (or -1 if no valid element exists).

Constraints

- $1 \leq T \leq 5$
- $1 \leq N, q \leq 10^5$
- $0 \leq arr[i], x, m \leq 10^9$

Sample Input / Output

Example 1

Input:

```
1
7
3 7 19 18 7 12 17
7
3 8
21 20
24 17
1 7
```

23 17

12 9

99 2

Output:

7

12

7

7

12

3

-1

Explanation

In the first query, $x = 3$ and $m = 8$. The array elements not exceeding 8 are 3, 7, 7. Their XORs with 3 are $3 \oplus 3 = 0$ and $3 \oplus 7 = 4$, so the best value is 7.

In the last query, $x = 99$ and $m = 2$. Every array element exceeds 2, so no candidate qualifies and the answer is -1.

Approach to Solve This Problem:

A brute-force check of every array element against every query gives $O(N \cdot q)$, which is too slow for $N, q \leq 10^5$. Instead, this problem is solved **offline** using a **binary trie built incrementally**, processing both array elements and queries together in sorted order of value.

1. **Sort array elements** in ascending order, keeping track of their original values.
2. **Sort queries** by m in ascending order, keeping track of their original index (to restore output order at the end).
3. **Two-pointer sweep:** Walk through the sorted queries in increasing order of m . For each query, first **insert into the trie** every array element that is $\leq m$ and hasn't been inserted yet (advance a pointer over the sorted array). This way, the trie always contains exactly the set of elements valid for the current query.
4. **Query the trie for maximum XOR:** Starting from the trie's root, for each bit of x from the most significant bit down to the least significant bit, greedily try to move into the child representing the **opposite bit** (since XOR-ing opposite bits gives 1, maximizing the result). If that child doesn't exist, fall back to the same-bit child.
5. If the trie is empty at query time (no elements $\leq m$ have been inserted yet), the answer is -1.
6. Store each answer at its original query index (since queries were sorted), then print all answers in original order.

Since $\text{arr}[i], x \leq 10^9 < 2^{30}$, using **30 bits** per number is sufficient for the trie depth.

Java Solution:

```
import java.util.*;

class Main {
    static final int BITS = 30; // since values  $\leq 10^9 < 2^{30}$ 

    static class TrieNode {
        TrieNode[] child = new TrieNode[2];
    }

    static TrieNode root;

    static void insert(int num) {
        TrieNode node = root;
        for (int b = BITS - 1; b >= 0; b--) {
            int bit = (num >> b) & 1;
            if (node.child[bit] == null) {
                node.child[bit] = new TrieNode();
            }
            node = node.child[bit];
        }
    }

    static int queryMaxXorValue(int x) {
        TrieNode node = root;
        int result = 0;
        for (int b = BITS - 1; b >= 0; b--) {
            int bit = (x >> b) & 1;
            int want = 1 - bit; // opposite bit maximizes XOR at this position
            if (node.child[want] != null) {
                result |= (want << b);
                node = node.child[want];
            } else {
                result |= (bit << b);
                node = node.child[bit];
            }
        }
        return result;
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int t = Integer.parseInt(sc.nextLine().trim());
        StringBuilder sb = new StringBuilder();
        while (t-- > 0) {
            int n = Integer.parseInt(sc.nextLine().trim());
            StringTokenizer st = new StringTokenizer(sc.nextLine());
            int[] arr = new int[n];
            for (int i = 0; i < n; i++) arr[i] = Integer.parseInt(st.nextToken());
            Arrays.sort(arr);

            int q = Integer.parseInt(sc.nextLine().trim());
            int[][] queries = new int[q][3]; // x, m, originalIndex
            for (int i = 0; i < q; i++) {
                StringTokenizer qt = new StringTokenizer(sc.nextLine());
                int x = Integer.parseInt(qt.nextToken());
                int m = Integer.parseInt(qt.nextToken());
                queries[i][0] = x;
                queries[i][1] = m;
                queries[i][2] = i;
            }
            // Sort queries by m ascending
            Arrays.sort(queries, (a, b) -> Integer.compare(a[1], b[1]));

            int[] ans = new int[q];
            root = new TrieNode();
            boolean anyInserted = false;
            int ptr = 0;
            for (int[] query : queries) {
                int x = query[0], m = query[1], idx = query[2];
                // Insert all array values <= m not yet inserted
                while (ptr < n && arr[ptr] <= m) {
                    insert(arr[ptr]);
                    anyInserted = true;
                    ptr++;
                }
                if (!anyInserted) {
                    ans[idx] = -1;
                } else {
                    ans[idx] = queryMaxXorValue(x);
                }
            }
            for (int i = 0; i < q; i++) {
                sb.append(ans[i]).append('\n');
            }
        }
        System.out.print(sb);
    }
}
```

Solution:

1. **Read input:** parse T test cases; for each, read N, the array, q, and all queries.
2. Sort the array ascending so elements can be inserted into the trie incrementally in increasing order of value.
3. Sort queries by m ascending while remembering each query's original index, since answers must ultimately be printed in the original input order.
4. **Two-pointer insertion:** as queries are processed in increasing m order, a pointer advances through the sorted array, inserting every element $\leq m$ into the binary trie exactly once — ensuring the trie always reflects precisely the valid candidate set for the current query.
5. **Trie-based max-XOR query:** for each query's x, walk the trie from the most significant bit downward, always preferring the child bit that is the opposite of x's current bit (this maximizes the XOR at each bit position greedily, which is optimal since higher bits dominate the final value).
6. **Handle the empty case:** if no elements have been inserted into the trie yet when a query is processed, output -1 for that query.
7. **Restore original order and print:** answers are stored by original query index, then printed in that same order at the end of each test case.

Complexity Analysis:

Time Complexity: $O(N)$

Sorting the array and queries takes $O(N \log N + q \log q)$. Each of the N insertions and q queries touches the trie for BITS = 30 levels, giving $O((N + q) \cdot 30)$ for the trie operations — overall near-linear in N and q.

Space Complexity: $O(N)$

The trie can have up to $N \times \text{BITS}$ nodes in the worst case (when array elements share few common prefixes), plus $O(N + q)$ for the sorted arrays and query bookkeeping.

